
Designing the Amazon Shopping App

From one program on one computer to thousands of programs on thousands of computers

Soumyajyoti Dutta

Teaching Assistant · Texas A&M University

THE QUESTION FOR TODAY

What happens when you tap **Buy** **Now?**

Race conditions

→ two shoppers, one last item

Caching and LRU

→ CDN edges and Redis

Producer–consumer

→ message queues between services

Today: the CSCE 313 ideas you already know, stretched across thousands of machines.

THE METHOD

Four steps for any system design problem

1

Clarify

What must it do? How fast, how available, how correct?

2

Estimate

Users, requests per second, storage, bandwidth.

3

Design

Boxes and arrows. Walk one request through them.

4

Stress-test

What breaks first? Fix it and name the trade-off.

Today follows these four steps in order. The same steps work in interviews and in your own projects.

STEP 1 · REQUIREMENTS

What the app must do, and how well

Functional

- Search and browse 500M products
- Product page: price, stock, delivery date
- A cart that follows you across devices
- Checkout and payment
- Orders: tracking, cancel, returns
- Reviews, recommendations, notifications
- Third-party seller listings

Non-functional

Need	Target
Availability	99.99%: about 52 minutes of downtime a year
Latency	p99 under 200–300 ms for browse and search
Consistency	Strong for stock and money; eventual elsewhere
Scale	10× normal peak on Prime Day, no redesign
Durability	Never lose a confirmed order or payment

THE KEY OBSERVATION

Different parts need different guarantees

AVAILABLE · MAY BE SLIGHTLY STALE

**Browse, search, cart, reviews,
recommendations**

A mistake is cheap: a stale review count, a merged cart.

EXACTLY CORRECT · MAY BE SLOWER

Inventory, orders, payments

A mistake is expensive: an oversold console, a double charge.

CAP theorem: when the network splits replicas apart, each piece of data must choose consistency or availability. We choose per component, not once for the whole system.

STEP 2 · ESTIMATION

The numbers decide the design

2B

page views a day (100M users × 20)

→ reads dominate everything

23k/s

average requests (÷ 86,400 seconds)

→ scale out, not up

115k/s

normal peak, 5× the average

→ Little's Law: ~5,750 in flight

200 : 1

reads to writes

→ cache hard; add read replicas

4 PB

of images downloaded a day (~370 Gbit/s)

→ a CDN is mandatory

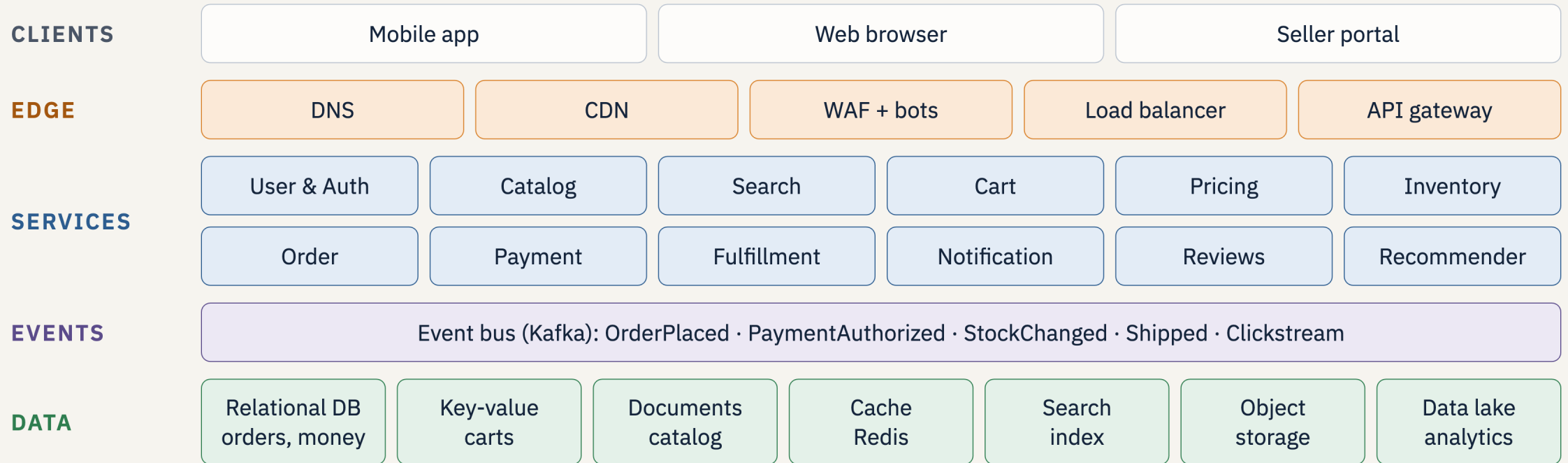
1,200/s

orders at an event peak

→ the small, consistent path

STEP 3 · HIGH-LEVEL DESIGN

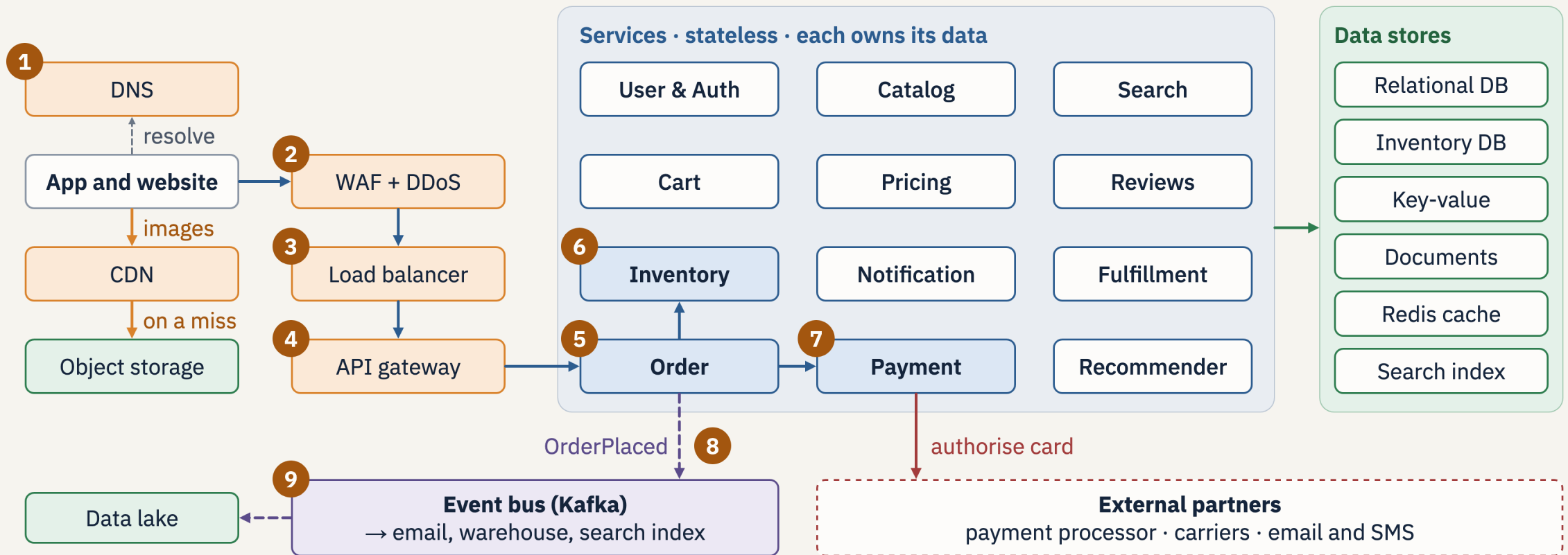
The reference architecture, top to bottom



Services are stateless and each owns its data. Cross-cutting: observability, service discovery, config and feature flags, secrets, CI/CD, autoscaling.

MASTER BLOCK DIAGRAM

The whole system on one page



Solid: the user waits · **Dashed:** after the reply · **Orange:** images via the CDN · **Red:** outside partner · **1-9:** the Buy Now steps

ONE REQUEST, END TO END

Tap Buy Now: nine steps

1 DNS

Points the phone at the nearest healthy region

2 TLS + WAF

Encrypts the connection; drops attacks and bots

3 Load balancer

Picks one healthy gateway out of many

4 API gateway

Checks the token, rate-limits, routes to Order

5 Order service

Checks the idempotency key; re-prices the cart

6 Inventory

Atomically reserves the items for 15 minutes

7 Payment

Authorises the card, reusing the same key

8 Confirm

Order CONFIRMED; publishes OrderPlaced

9 After the reply

Email, warehouse, analytics, recommendations

Steps 5–8 (blue): the critical path the user waits for, about 1–2 s. **Step 9 (dashed):** asynchronous, after the response.

Get each request to a healthy server, fast

Component	Job	Breaks when... → fix
DNS	Send users to the nearest healthy region	Cached answers slow failover → short TTLs, client retries
CDN	Serve images and scripts from nearby edges	Stale files after a deploy → content-hashed file names
WAF	Block attacks, bots, scalpers	Blocks real shoppers → test new rules in count-only mode
Load balancer	Spread load, route around dead servers	Health check too deep removes every server → fail open
API gateway	Auth, rate limiting, routing	Business logic creeps in → keep it thin

Why many small services, not one big program

	Monolith	Microservices
Deploy	Everything at once	Each service on its own
Scale	Everything together	Only the busy part
Failure	One leak can crash all	Contained, if designed well
Data	One DB, easy transactions	DB per service; sagas, events
Debug	One stack trace	Distributed tracing

WIDELY REPORTED

Amazon moved from a monolith to services in the early 2000s: small teams, each owning a service, talking only through APIs.

CSCE 313 LINK

A service is a process with a private address space: no shared memory, only messages.

The cart picks availability over consistency

- Key-value store: key = user, value = items
- Replicated 3×; "Add to cart" always succeeds
- Not a reservation: price and stock are re-checked at checkout

DOCUMENTED HISTORY · DYNAMO, SOSP 2007

Amazon built its store so the cart stays writeable during failures. Diverging copies are merged, and the paper notes a deleted item can reappear.

Phone, offline
adds Headphones

Laptop
removes Charger



Merged cart
Headphones in. Charger stays out, because a tombstone remembers the removal.

Without tombstones the merge would bring the Charger back.

The last PS6: two buyers, one console

Step	Alice's server	Bob's server
1	reads stock = 1	reads stock = 1
2	$1 \geq 1$, so OK	$1 \geq 1$, so OK
3	writes 0	writes 0

Result: two orders, one console. A lost update, the same race as two threads incrementing one counter.

THE FIX: ONE ATOMIC STATEMENT

```
UPDATE inventory
SET reserved = reserved + 1
WHERE sku_id = 'PS6'
    AND on_hand - reserved ≥ 1;

-- 1 row updated → you got it
-- 0 rows updated → sold out
```

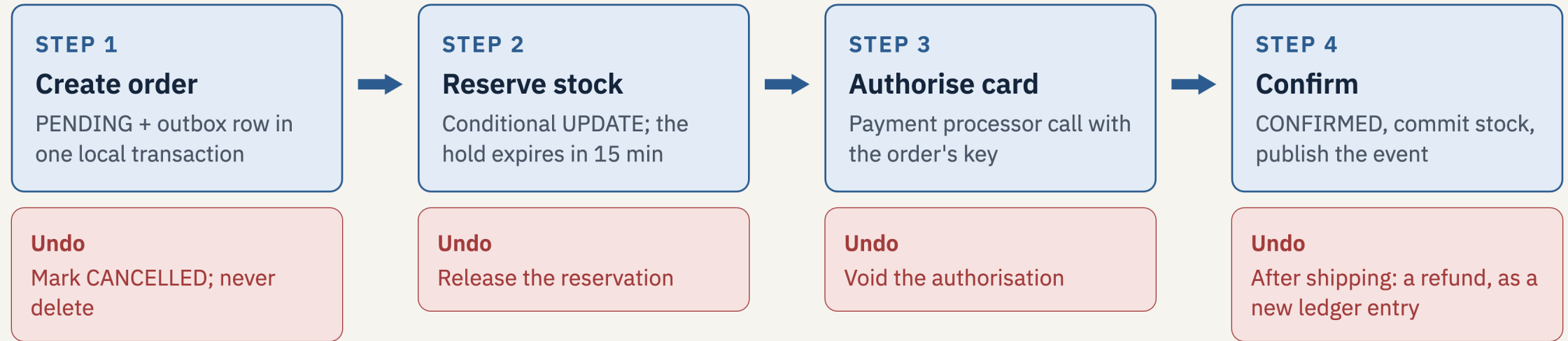
Why not a mutex in the Order service? Alice and Bob hit different machines. The lock must live where both requests meet: the database row.

Ways to fix the race, and what each costs

Approach	Correct?	Good when	Weak spot
Read, check, write	No	Never	Lost updates
Row lock (FOR UPDATE)	Yes	Moderate traffic	Buyers queue on one lock
Version check	Yes	Rare conflicts	Retry storms when busy
Conditional UPDATE	Yes	The default	A hot row still serialises
Reservation + 15 min TTL	Yes	Always, on top	Needs an expiry sweeper
Redis token gate + DB	Yes	Flash sales	Gate and DB must reconcile

CSCE 313: row lock = mutex · conditional UPDATE = compare-and-swap · lock several SKUs in sorted order to avoid deadlock

Checkout is a saga: local steps, each with an undo



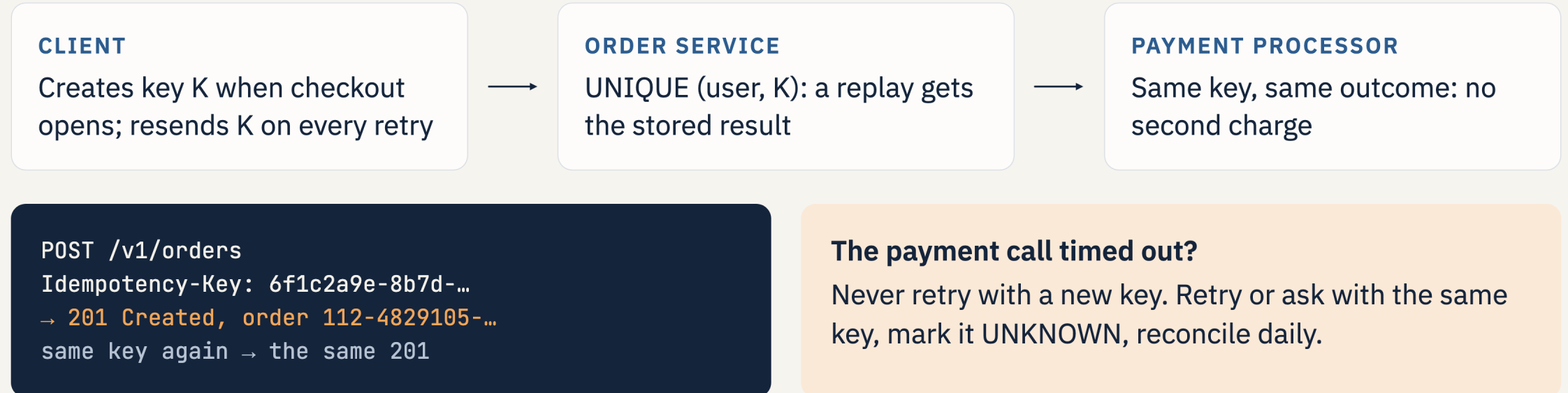
Why not two-phase commit (2PC)?

It blocks if the coordinator dies, holds locks across slow calls, and card networks won't join in.

Order the steps on purpose

Likely failure first (stock), hardest undo last (money). Logged state lets a crash resume.

Never charge twice: one key, end to end



Idempotent: doing it twice has the same effect as doing it once, like pressing an elevator button again.

The right store for each kind of data

Data	Store	Why this one
Orders, payments	Relational, sharded by user	ACID, constraints, exact money
Inventory	SQL, conditional writes	Atomic decrement, never oversell
Cart, sessions	Key-value store	One key per user, always writeable
Product catalog	Document store + cache	Flexible attributes, huge read load
Search	Inverted index	Relevance, facets, autocomplete
Images, video	Object storage + CDN	Cheap, durable, served from the edge
Events	Distributed log (Kafka)	Ordered per key, replayable
Analytics	Data lake + warehouse	Big scans, kept off production

With 200 reads per write, caching pays for itself

Cache-aside, the default

1. Read the cache
2. Miss? Read the database
3. Store the row in the cache with a TTL
4. On a write: update the DB, then delete the key

Levels: device → CDN → in-process → Redis → database buffer pool

CSCE 313: LRU replacement, write-through vs write-back, cold-cache misses. Same ideas, keys instead of pages.

Stale data

TTL as a safety net; delete on write; version numbers or leases stop old values winning a race

Stampede

A hot key expires and 10,000 requests hit the DB at once. Let one request refresh while others wait or get the stale value; add jitter to TTLs

Hot key

One viral product overloads one cache node. A few seconds of in-process cache; copies of the key on several nodes

Do only what the user waits for; queue the rest

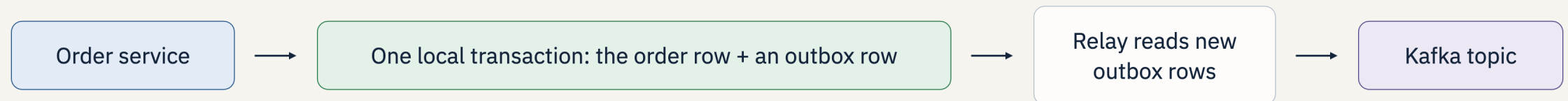
The user waits for

- Authenticate and re-price
- Reserve stock, authorise the card
- Write the order

Happens after, from the queue

- Email, push, warehouse, shipping
- Search index and analytics updates
- Recommendations

THE DUAL-WRITE FIX: TRANSACTIONAL OUTBOX



A crash means the event is published again (a duplicate), never lost. Consumers skip event IDs they have already processed.

Replicas for copies, shards for size

Replication

- The leader takes writes; followers serve reads
- Lag can hide your own write: "my review vanished"
- Fix: read-your-writes, so the author reads from the leader
- Quorum: $R + W > N$, e.g. $N = 3, W = 2, R = 2$

Sharding

Data	Shard key
Orders, carts	user_id
Inventory	sku_id
Reviews	product_id

Pick the key the main query uses ("my orders" hits one shard).
Consistent hashing moves only about $1/N$ of keys when a node joins.
Watch for hot shards.

One slow service must not sink the site

Anatomy of a cascading failure

- 14:00:00 Reviews database slows from 5 ms to 5 s
- 14:00:30 Product-page servers wait on Reviews with 30 s timeouts
- 14:01:30 Thread pools fill up; every product page stalls
- 14:02:00 Retries multiply load; health checks eject servers
- 14:03:00 **Product page down site-wide, over a non-essential widget**

CSCE 313: thread-pool exhaustion, with every worker blocked on a slow read().

Timeouts

100 ms on a non-essential call, not a 30 s default

Retries

Backoff with jitter, at one layer.
3 layers × 3 tries = 27× load

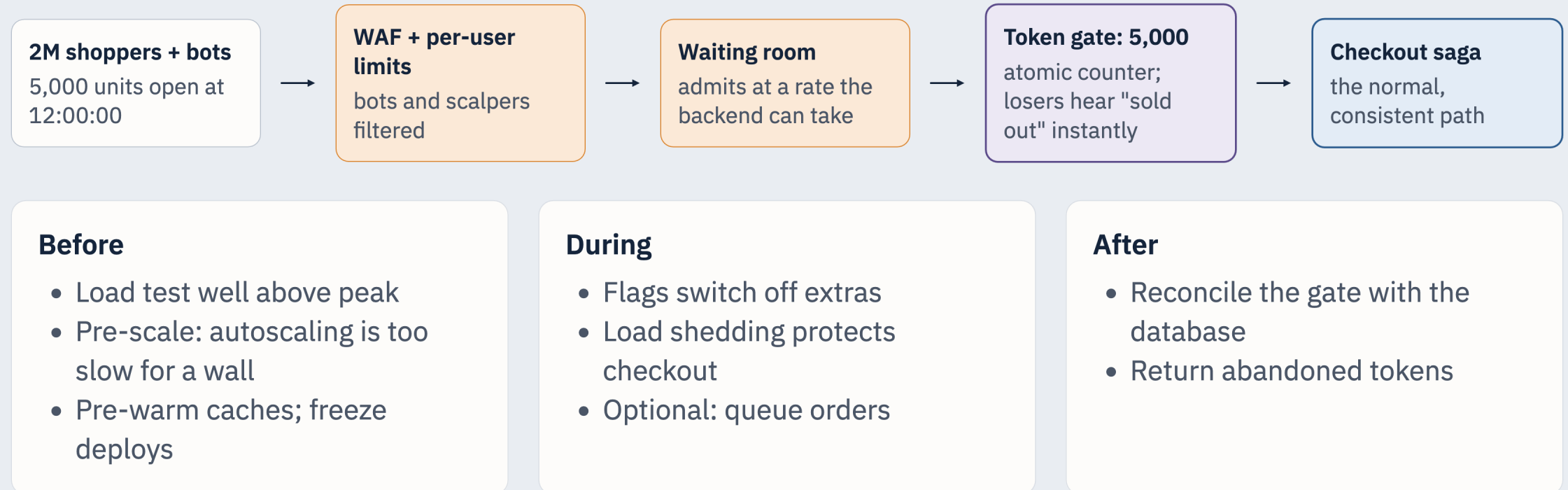
Circuit breaker

After repeated failures, fail fast with a fallback; probe again later

Bulkheads, shedding

A separate pool per dependency; drop recommendations before checkout

Shrink the crowd before it reaches the database



WRAP-UP

Every choice was a trade-off

Decision	Choice	We gain	We pay
Architecture	Microservices	Independent teams	Network failures
Cart	AP key-value	Always add to cart	Rare merge conflicts
Inventory	Strong consistency	Never oversell	Hot-item contention
Checkout	Saga	Recoverable steps	No isolation; undo logic
Reads	Caching	Speed, DB offload	Staleness
Side effects	Events + outbox	Fast checkout	Delay, duplicates
Availability	Multi-region	Survive a region loss	Cost, replication lag

You already know this, at a smaller scale

In CSCE 313	In the Amazon design
Race condition, critical section	The last-item inventory race
Atomic compare-and-swap	Conditional UPDATE, version checks
Mutex, deadlock, lock ordering	Row locks; lock SKUs in sorted order
Bounded buffer, producer–consumer	Message queues and back-pressure
Thread pools	Server worker pools, bulkheads
CPU caches, LRU, write-back	CDN, Redis, eviction policies
Journaling, write-ahead log	Database WAL, Kafka log, outbox
Processes and IPC	Microservices, RPC, events
Floating-point representation	Money stored as integer cents

IF YOU REMEMBER ONE THING

Be available where mistakes are cheap.

Be consistent where they are expensive.

Cart, browsing, reviews → available. Inventory, orders, payments → consistent. Now it's your turn to design.